

Programming in Python for Friends and Relations, Part 12

Do You Speak my Language?



It is worth adding TTS to your applications, for no reason other than to attract attention.

"Grandpa, what was life like before computers got so small that they became invisible?" This caption of a cartoon sums up our predicament. How do we communicate with applications on our ever-shrinking computers? On the smaller machines, even familiar applications could look different and behave oddly. We may need help to use the different interface. However, reading a help document on a cramped Netbook or a smartphone screen is not easy. Wouldn't it be nicer if the 'mouse over' text or a help message could be read out to us instead?

Efforts at text-to-speech have been around long before computers were created (see en.wikipedia.org/wiki/Text_to_speech). Getting the computer to speak in a nice voice has proven to be a remarkably hard

task. Many Interactive Voice Response (IVR) systems get around the problem by recording phrases spoken by people and playing back the appropriate sequence of wave files. This is clearly feasible if the size of the required vocabulary is small.

If you are willing to compromise and accept a voice that is comprehensible, even if robotic, there are a few options. The eSpeak (espeak.sourceforge.net) system should be the first to be explored, as it is widely used for its accessibility features and is the platform the OLPC/Sugar project uses. It is small and has support for a fair number of languages.

Getting started

Python provides a wrapper for using the Speech Dispatcher server, which is generic for text to speech (TTS) applications. It can